

Python Course for Physicists (And everyone else!)

Chunk 12: Scipy & Regressions

By Jonathan Bollig

08.10.2025

Matplotlib: rcParams

- rcParams: Set style of plots for complete document:

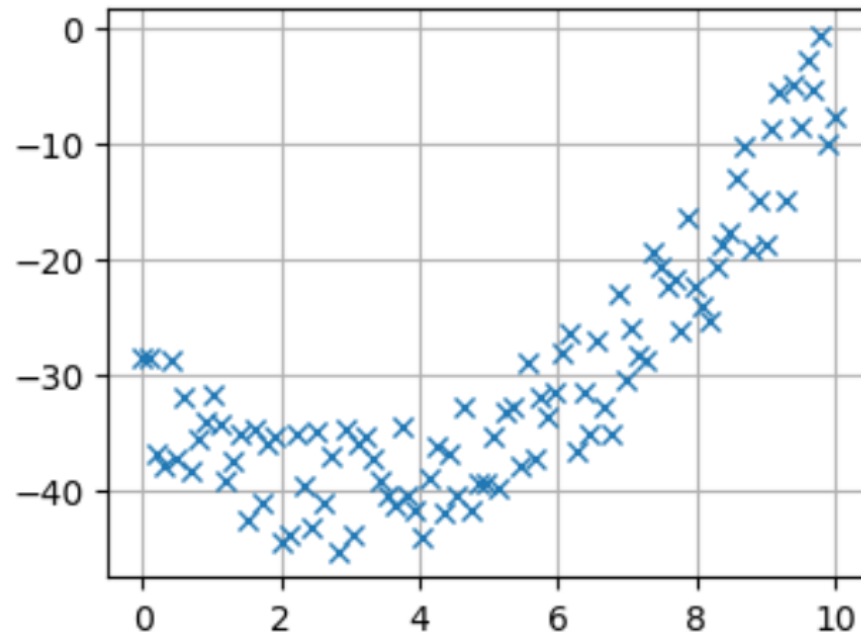
```
%matplotlib widget
import matplotlib.pyplot as plt
plt.rcParams["figure.figsize"] = (4,3)
plt.rcParams["axes.grid"] = True
```

- Complete set of parameters [here](#).

Live Coding: Visualizing Noisy Data

- `plt.plot(x, y, marker="x", linestyle="")`

```
plt.plot(x, y_measurement, marker="x", linestyle="")  
plt.show()
```



Finding the Function: Regression

- Usually: We have an idea of the “nature” of our data.
→ Example shown: “looks like a quadratic function”
- But: We don’t know the constants describing the polynomial
- Two options:
 1. `np.polyfit(x, y, order)` → returns the coefficients
 2. General function definition → `scipy.optimize.curve_fit(...)`

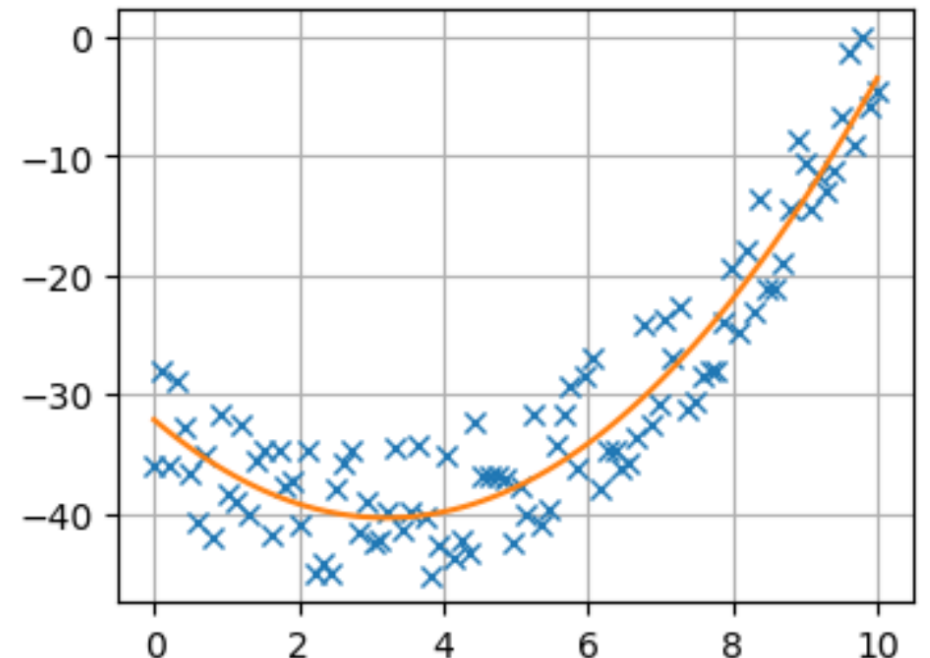
Live Coding: Numpy polyfit

- polyfit finds coefficients.
- Coefficients can be used to create a fitted function
- Plot the fitted function

```
coeffs = np.polyfit(x, y_measurement, 2)
print(coeffs)
y_fit = coeffs[0] * x**2 + coeffs[1] * x + coeffs[2]

plt.plot(x, y_measurement, marker="x", linestyle="")
plt.plot(x, y_fit)
# plt.plot(x, y_theory);
```

```
[ 0.80085884 -5.14075265 -32.11890331]
[<matplotlib.lines.Line2D at 0x1a30dd41090>]
```



scipy (constants)



- scipy → scientific python
- Tons of useful functions
- One example: scipy.constants:

```
import scipy.constants as c  
print(c.c)
```

```
299792458.0
```

Live Coding: scipy curve_fit

- Need to create a python function, that describes our mathematical function
→ Order matters
- Create coefficients & covariance
 - Covariance can be used to calculate confidence intervals
- If convergence fails: Give initial guess with optional p0 parameter

1. x

2. parameter

```
def second_op(x, c1, c2, c3):  
    return c1 * x**2 + c2 * x + c3
```

```
import scipy  
  
def second_op(x, c1, c2, c3):  
    return c1 * x**2 + c2 * x + c3  
  
coeffs, cv = scipy.optimize.curve_fit(second_op, x, y_measurement)  
print(coeffs)  
  
[ 0.80085884 -5.14075261 -32.11890338]
```

Good Luck with your Tasks 😊

